

Four Game

Yusupov Boburjon

Neptun: YTAJDI

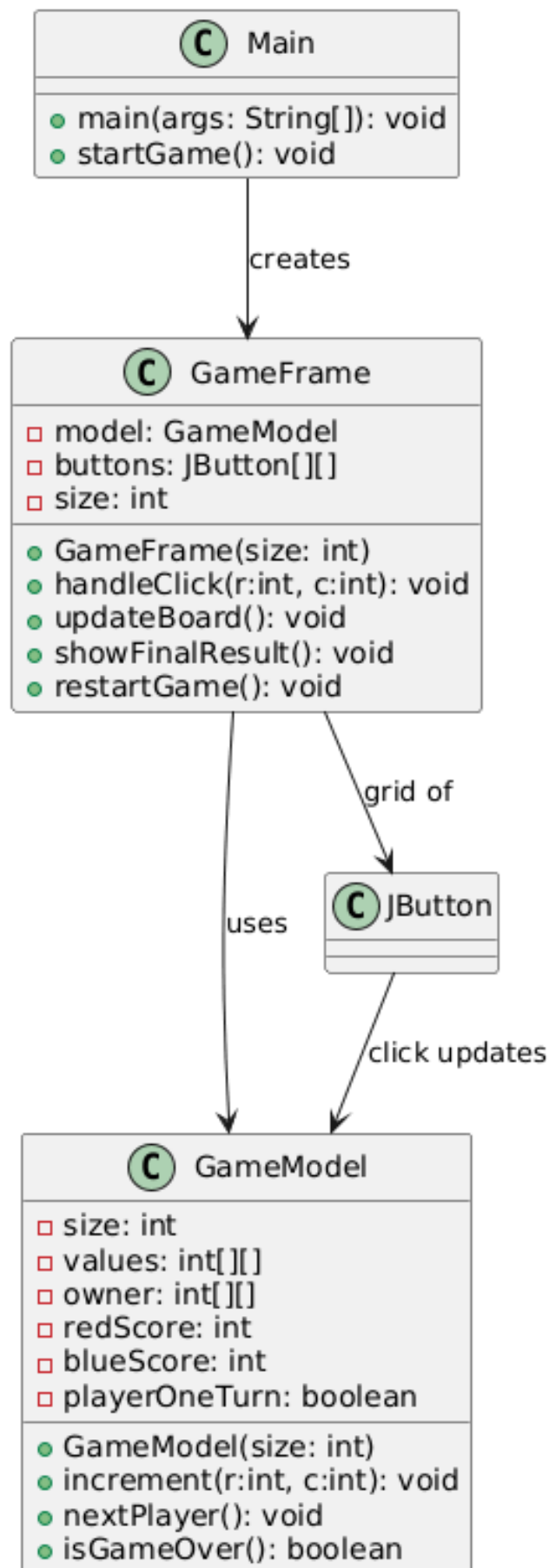
November 1, 2025

1 Description of the Exercise

This a two-player game is played on a board consists of $n \times n$ fields, where each field contains a value between 0 and 4. Initially, all the fields contain the value of 0. If a player chooses a field, then the value of the field and its neighbours incremented by one (if the value is less than 4). The player's score represents how many fields did he make to have the value of 4. If a value of a field reaches 4, then the field is colorized with the color of the actual player (red or blue). The game ends, when all fields have the value of 4. The player having the higher score wins. Implement this game, and let the board size be selectable (3x3, 5x5, 7x7). The game should recognize if it is ended, and it has to show in a message box which player won. After this, a new game should be started automatically.

2 Class Diagram

Reach 4 Game - Simplified UML (No Getters/Setters)



2.1 Class Overview

The application follows an object-oriented design with three main classes:

- **Main:** Entry point and game launcher
- **GameFrame:** User interface and game controller
- **GameModel:** Game logic and state management

3 Class and Method Descriptions

3.1 Main Class

The `Main` class serves as the entry point for the application and handles game initialization.

3.1.1 Fields

- `testBoardSize: Integer` – Static field for testing purposes to force a specific board size
- `testMode: boolean` – Static flag to enable test mode (disables popups)

3.1.2 Methods

- `main(args: String[]): void`
 - **Purpose:** Application entry point
 - **Description:** Invokes the `startGame()` method on the Event Dispatch Thread using `SwingUtilities.invokeLater()`
 - **Parameters:** Command-line arguments (not used)
 - **Returns:** void
- `startGame(): void`
 - **Purpose:** Initializes and starts a new game
 - **Description:** Displays a dialog for board size selection (3x3, 5x5, or 7x7), creates a new `GameFrame` instance with the selected size, and handles test mode for automated testing
 - **Parameters:** None
 - **Returns:** void

3.2 GameModel Class

The `GameModel` class encapsulates the game logic and state. It maintains the board values, ownership, scores, and player turn information.

3.2.1 Fields

- `size: int` – The dimension of the square board (3, 5, or 7)
- `values: int[][]` – 2D array storing the current value (0-4) of each field
- `owner: int[][]` – 2D array storing the owner of each field (0=none, 1=red, 2=blue)
- `redScore: int` – Count of fields that reached 4 by the red player
- `blueScore: int` – Count of fields that reached 4 by the blue player
- `playerOneTurn: boolean` – Flag indicating whether it's player one's turn (true=red, false=blue)

3.2.2 Methods

- `GameModel(size: int)`
 - **Purpose:** Constructor
 - **Description:** Initializes the game model with the specified board size, creates the values and owner arrays, and sets all initial values to 0
 - **Parameters:** size – The dimension of the board
 - **Returns:** N/A (constructor)
- `increment(r: int, c: int): void`
 - **Purpose:** Core game logic for field increment
 - **Description:** Increments the value of the field at position (r, c) and all its neighbors (up, down, left, right) by 1, if their values are less than 4. If a field reaches 4, it assigns ownership to the current player and increments their score. Neighbors outside the board boundaries are ignored.
 - **Parameters:** r – Row index; c – Column index
 - **Returns:** void
- `nextPlayer(): void`
 - **Purpose:** Switch turns
 - **Description:** Toggles the `playerOneTurn` flag to switch between red and blue players
 - **Parameters:** None
 - **Returns:** void
- `isGameOver(): boolean`
 - **Purpose:** Check game end condition
 - **Description:** Checks if all fields on the board have reached the value of 4. Returns true if the game has ended, false otherwise.
 - **Parameters:** None
 - **Returns:** true if game is over, false otherwise

3.3 GameFrame Class

The `GameFrame` class extends `JFrame` and serves as the graphical user interface and controller for the game.

3.3.1 Fields

- `model: GameModel` – The game model instance
- `buttons: JButton[][]` – 2D array of buttons representing the game board
- `testMode: boolean` – Flag to disable popups during testing

3.3.2 Methods

- `GameFrame(size: int)`
 - **Purpose:** Constructor
 - **Description:** Creates the game window with a grid layout of buttons. Each button is configured with initial text "0", appropriate size (100x100 pixels), and an action listener. The window is sized to fit the board and centered on screen.
 - **Parameters:** size – The dimension of the board
 - **Returns:** N/A (constructor)
- `handleClick(r: int, c: int): void`
 - **Purpose:** Event handler for button clicks
 - **Description:** Called when a button is clicked. Calls `model.increment(r, c)` to update game state, calls `updateBoard()` to refresh the display, checks if the game is over, and if so, shows the final result and restarts the game. Otherwise, switches to the next player.
 - **Parameters:** r – Row of clicked button; c – Column of clicked button
 - **Returns:** void
- `updateBoard(): void`
 - **Purpose:** Refresh the visual display
 - **Description:** Updates all button texts to reflect current field values. If a field has reached 4, its button background is colorized (red or blue) based on ownership.
 - **Parameters:** None
 - **Returns:** void
- `showFinalResult(): void`
 - **Purpose:** Display game result
 - **Description:** Shows a message dialog with the winner (or "Draw" message) and the final scores of both players. Only called when not in test mode.
 - **Parameters:** None
 - **Returns:** void
- `getWinnerMessage(): String`
 - **Purpose:** Determine the winner
 - **Description:** Compares red and blue scores and returns the appropriate winner message ("Red Player Wins!", "Blue Player Wins!", or "It's a Draw!")
 - **Parameters:** None
 - **Returns:** Winner message string
- `restartGame(): void`
 - **Purpose:** Restart the game
 - **Description:** Disposes the current game window and calls `Main.startGame()` to create a new game (unless in test mode)
 - **Parameters:** None
 - **Returns:** void

4 Events and Event Handlers

This section describes the connections between user actions (events) and the corresponding event handler methods.

4.1 Event Flow Diagram

Event	Event Handler	Description
Application Start	<code>Main.main()</code>	Entry point that initiates the game
Game Initialization	<code>Main.startGame()</code>	Displays board size selection dialog and creates <code>GameFrame</code>
Board Size Selection	<code>Main.startGame()</code>	User selects board size (3x3, 5x5, or 7x7) from dialog
Button Click	<code>GameFrame.handleClick(r, c)</code>	User clicks a field button to make a move
Field Increment	<code>GameModel.increment(r, c)</code>	Updates field values and scores based on game rules
Display Update	<code>GameFrame.updateBoard()</code>	Refreshes button texts and colors after each move
Game Over Check	<code>GameModel.isGameOver()</code>	Checks if all fields have reached value 4
Game End	<code>GameFrame.showFinalResult()</code>	Displays winner message and final scores
Game Restart	<code>GameFrame.restartGame()</code>	Closes current game and starts a new one

Table 1: Event-Handler Connections

4.2 Detailed Event Descriptions

4.2.1 Button Click Event

- **Trigger:** User clicks on any field button
- **Event Source:** `JButton` (from the buttons array)
- **Event Listener:** `ActionListener` attached in the `GameFrame` constructor
- **Handler Method:** `handleClick(r, c)`
- **Actions Performed:**
 1. Calls `model.increment(r, c)` to update game state
 2. Calls `updateBoard()` to refresh the visual display
 3. Checks if the game is over using `model.isGameOver()`
 4. If game over: shows result and restarts game
 5. If not over: switches to next player using `model.nextPlayer()`

4.2.2 Window Creation Event

- **Trigger:** `GameFrame` constructor is called
- **Event Source:** `Main.startGame()` method
- **Handler Method:** `GameFrame(size)`
- **Actions Performed:**
 1. Creates a new `GameModel` instance
 2. Sets up the window with `GridLayout`
 3. Creates and configures all button instances
 4. Attaches `ActionListeners` to buttons
 5. Makes the window visible

4.2.3 Dialog Events

- **Board Size Selection Dialog:**
 - **Trigger:** `Main.startGame()` is called
 - **Event Source:** `JOptionPane.showInputDialog()`
 - **Handler:** Switch statement in `startGame()`
 - **Action:** Determines board size based on user selection
- **Game Over Message Dialog:**
 - **Trigger:** All fields reach value 4
 - **Event Source:** `showFinalResult()` method
 - **Handler:** `JOptionPane.showMessageDialog()`
 - **Action:** Displays winner and final scores

4.3 Event Sequence Flow

The typical sequence of events during a game session:

1. **Startup:** `main()` → `startGame()` → Board size dialog
2. **Initialization:** `GameFrame` constructor → `GameModel` constructor
3. **Gameplay Loop** (repeated):
 - User clicks button
 - `handleClick()` called
 - `model.increment()` updates state
 - `updateBoard()` refreshes display
 - `model.nextPlayer()` switches turns
4. **Game End:**
 - `isGameOver()` returns true
 - `showFinalResult()` displays message
 - `restartGame()` initiates new game
5. **Restart:** Returns to step 1 (board size selection)

5 Implementation Details

5.1 Object-Oriented Design

The application demonstrates several OOP principles:

- **Encapsulation:** The `GameModel` class encapsulates all game logic and state with private fields and public methods (getters/setters)
- **Separation of Concerns:** Clear separation between model (`GameModel`), view (`GameFrame`), and controller (event handlers)
- **Abstraction:** Game logic is abstracted away from UI concerns
- **Inheritance:** `GameFrame` extends `JFrame` to leverage Swing functionality

5.2 User Interface

- **Board Size Adaptation:** The window automatically resizes to fit boards of different sizes (3x3, 5x5, 7x7)
- **Visual Feedback:**
 - Button text shows current field values (0-4)
 - Background color indicates field ownership (red/blue)
 - Large, readable font (Arial Bold, 24pt)
- **User-Friendly Features:**
 - Clear board size selection dialog
 - Centered window positioning
 - Informative end-game messages
 - Automatic game restart

5.3 Testing Support

The implementation includes testing features:

- `testMode` flag to disable popups during automated tests
- `testBoardSize` to force specific board sizes in tests
- `TestGameFrame` subclass to override restart behavior in tests
- Public getter methods for test verification

6 Conclusion

The Four Game successfully implements all required features:

- Two-player gameplay with turn-based mechanics
- Increment logic affecting neighbors

- Scoring system based on reaching value 4
- Visual distinction between players (color coding)
- Multiple board sizes (3x3, 5x5, 7x7)
- Automatic game-over detection
- Winner announcement via message box
- Automatic game restart
- User-friendly interface with adaptive window sizing

The application demonstrates proper object-oriented design with clear class responsibilities, encapsulation, and separation of concerns between the model, view, and control logic.